

Diwali Sales Analysis

Exploratory Data Analysis using Python

Tools Used: Python, Pandas, NumPy, Matplotlib, Seaborn

Dataset: Diwali Sales Dataset (11,251 records)

Project Objective

The objective of this project is to analyze Diwali sales data using Python to understand customer purchasing behavior, identify sales trends, and generate business insights through exploratory data analysis (EDA).

Dataset Description

The dataset contains retail transaction records, including customer demographics, product categories, order details, purchase amount, occupation, state, and marital status. It is used to analyze customer buying patterns during the Diwali shopping season.

Import Libraries

```
In [7]: # import python libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt           # visualizing data
%matplotlib inline
import seaborn as sns
```

Loading the Dataset

```
In [8]: # import csv file
df = pd.read_csv('Diwali Sales Data.csv', encoding= 'unicode_escape')

In [9]: df.shape
```

Out[9]: (11251, 15)

In [10]: `df.head()`

Out[10]:

	User_ID	Cust_name	Product_ID	Gender	Age Group	Age	Marital_Status	
0	1002903	Sanskriti	P00125942	F	26-35	28	0	Mah
1	1000732	Kartik	P00110942	F	26-35	35	1	Andhra
2	1001990	Bindu	P00118542	F	26-35	35	1	Uttar
3	1001425	Sudevi	P00237842	M	0-17	16	0	Ka
4	1000588	Joni	P00057942	M	26-35	28	1	

In [11]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11251 entries, 0 to 11250
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User_ID                11251 non-null  int64
1   Cust_name              11251 non-null  object
2   Product_ID             11251 non-null  object
3   Gender                 11251 non-null  object
4   Age Group              11251 non-null  object
5   Age                    11251 non-null  int64
6   Marital_Status         11251 non-null  int64
7   State                  11251 non-null  object
8   Zone                   11251 non-null  object
9   Occupation             11251 non-null  object
10  Product_Category       11251 non-null  object
11  Orders                 11251 non-null  int64
12  Amount                 11239 non-null  float64
13  Status                  0 non-null      float64
14  unnamed1                0 non-null      float64
dtypes: float64(3), int64(4), object(8)
memory usage: 1.3+ MB
```

The dataset was successfully imported and inspected to understand its structure, dimensions, and available features.

Data Cleaning & Preprocessing

In [12]: `#drop unrelated/blank columns`
`df.drop(['Status', 'unnamed1'], axis=1, inplace=True)`

In [13]: `#check for null values`
`pd.isnull(df).sum()`

```
Out[13]: User_ID          0
         Cust_name       0
         Product_ID      0
         Gender          0
         Age Group       0
         Age             0
         Marital_Status  0
         State           0
         Zone            0
         Occupation      0
         Product_Category 0
         Orders          0
         Amount          12
         dtype: int64
```

```
In [14]: # drop null values
         df.dropna(inplace=True)
```

```
In [15]: # change data type
         df['Amount'] = df['Amount'].astype('int')
```

```
In [16]: df['Amount'].dtypes
```

```
Out[16]: dtype('int64')
```

```
In [17]: df.columns
```

```
Out[17]: Index(['User_ID', 'Cust_name', 'Product_ID', 'Gender', 'Age Group', 'Age',
               'Marital_Status', 'State', 'Zone', 'Occupation', 'Product_Category',
               'Orders', 'Amount'],
              dtype='object')
```

```
In [19]: # describe() method returns description of the data in the DataFrame (i.e.
         df.describe())
```

```
Out[19]:
```

	User_ID	Age	Marital_Status	Orders	Amount
count	1.123900e+04	11239.000000	11239.000000	11239.000000	11239.000000
mean	1.003004e+06	35.410357	0.420055	2.489634	9453.610553
std	1.716039e+03	12.753866	0.493589	1.114967	5222.355168
min	1.000001e+06	12.000000	0.000000	1.000000	188.000000
25%	1.001492e+06	27.000000	0.000000	2.000000	5443.000000
50%	1.003064e+06	33.000000	0.000000	2.000000	8109.000000
75%	1.004426e+06	43.000000	1.000000	3.000000	12675.000000
max	1.006040e+06	92.000000	1.000000	4.000000	23952.000000

```
In [20]: # use describe() for specific columns
         df[['Age', 'Orders', 'Amount']].describe()
```

Out [20]:

	Age	Orders	Amount
count	11239.000000	11239.000000	11239.000000
mean	35.410357	2.489634	9453.610553
std	12.753866	1.114967	5222.355168
min	12.000000	1.000000	188.000000
25%	27.000000	2.000000	5443.000000
50%	33.000000	2.000000	8109.000000
75%	43.000000	3.000000	12675.000000
max	92.000000	4.000000	23952.000000

Data Cleaning Summary

- Removed irrelevant columns.
- Identified and removed records containing missing values.
- Converted the Amount column to integer format.
- Verified data types and dataset consistency before analysis.

Exploratory Data Analysis

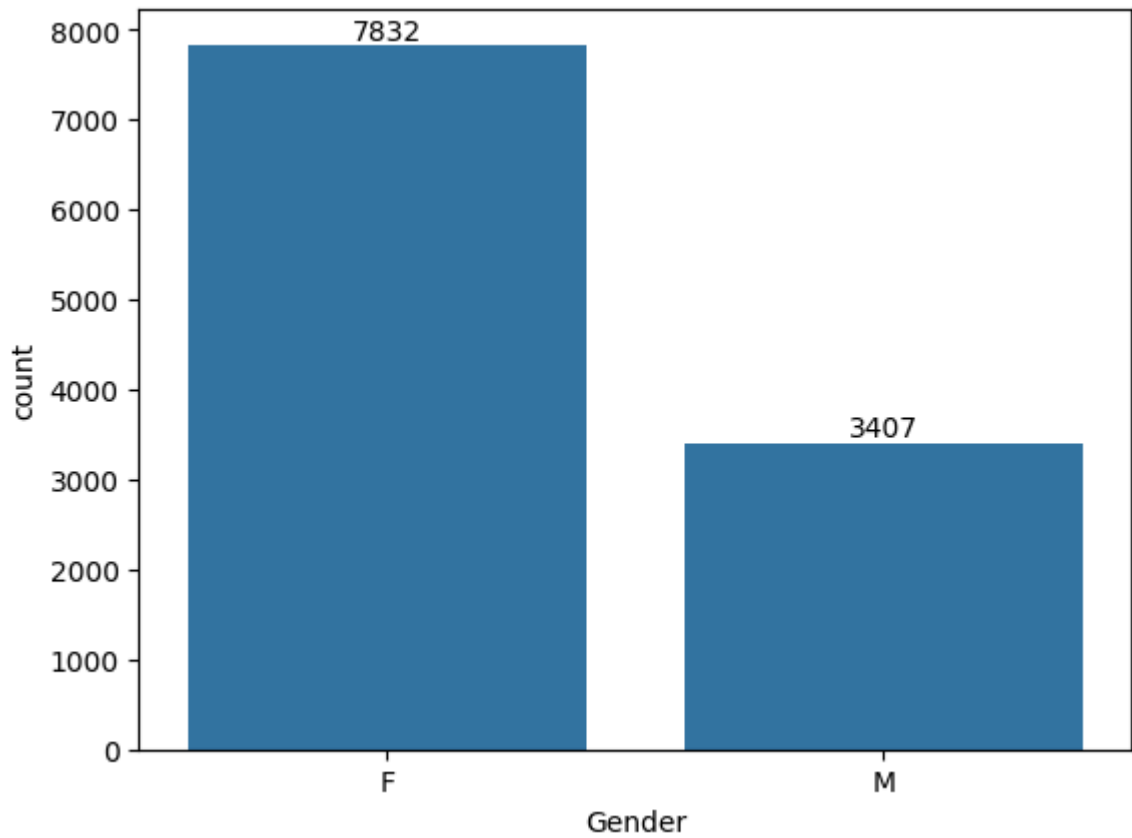
The following analysis explores customer demographics, purchasing behavior, and sales performance using statistical summaries and visualizations.

Gender Analysis

```
In [21]: #Plotting a bar chart for Gender and it's count.

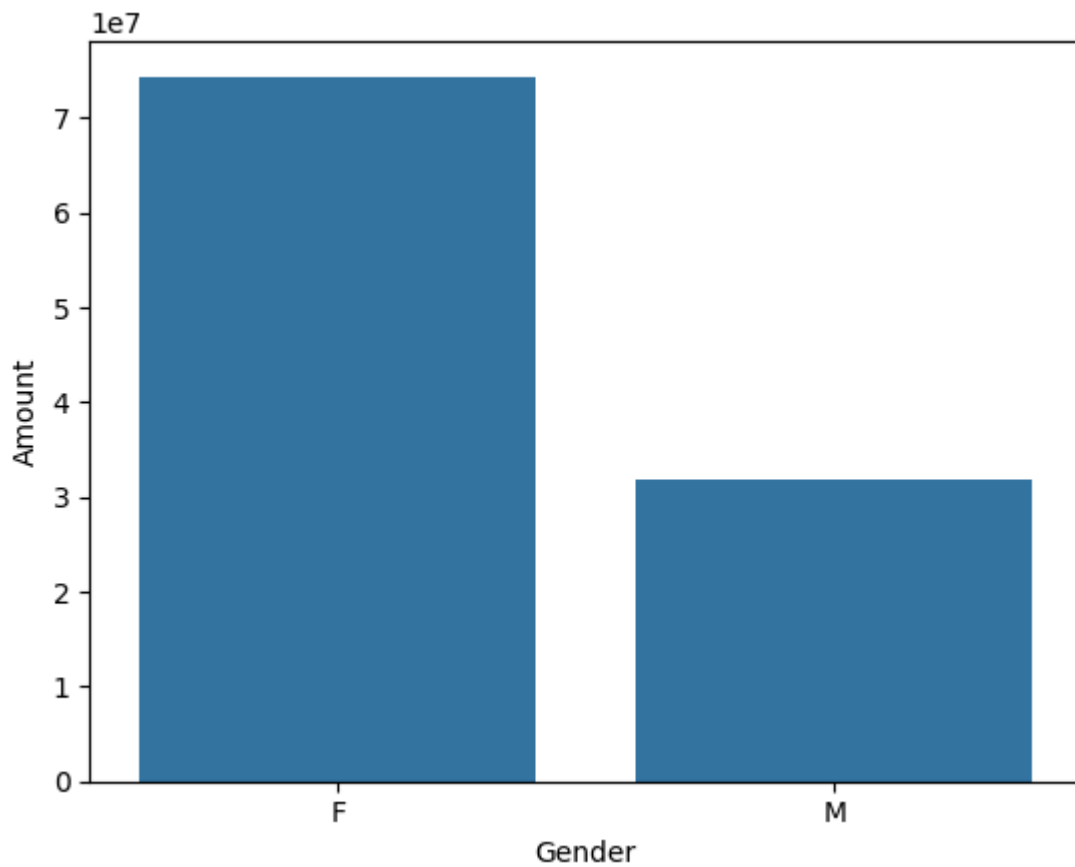
ax = sns.countplot(x = 'Gender',data = df)

for bars in ax.containers:
    ax.bar_label(bars)
```



```
In [22]: #Plotting a bar chart for gender vs total amount
sales_gen = df.groupby(['Gender'], as_index=False)['Amount'].sum().sort_v
sns.barplot(x = 'Gender',y= 'Amount' ,data = sales_gen)
```

```
Out[22]: <Axes: xlabel='Gender', ylabel='Amount'>
```

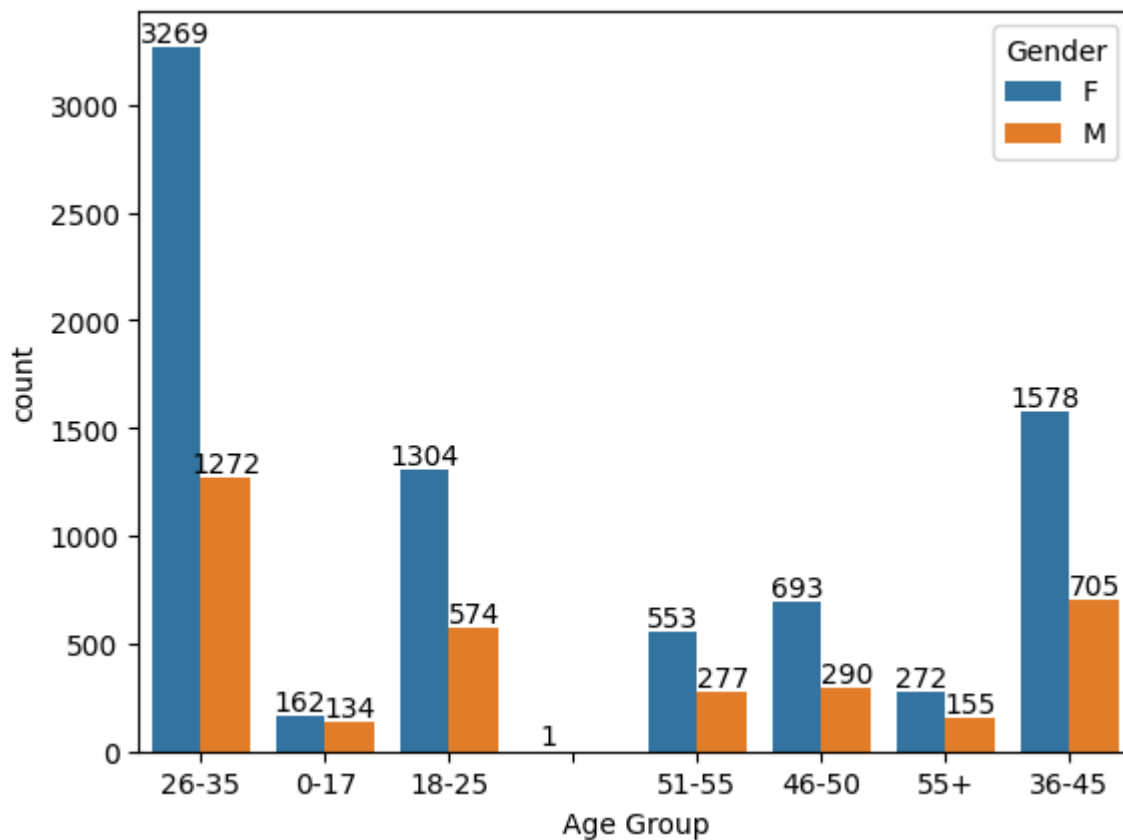


Observation

Female customers account for a larger share of purchases and contribute higher overall sales compared to male customers.

Age Analysis

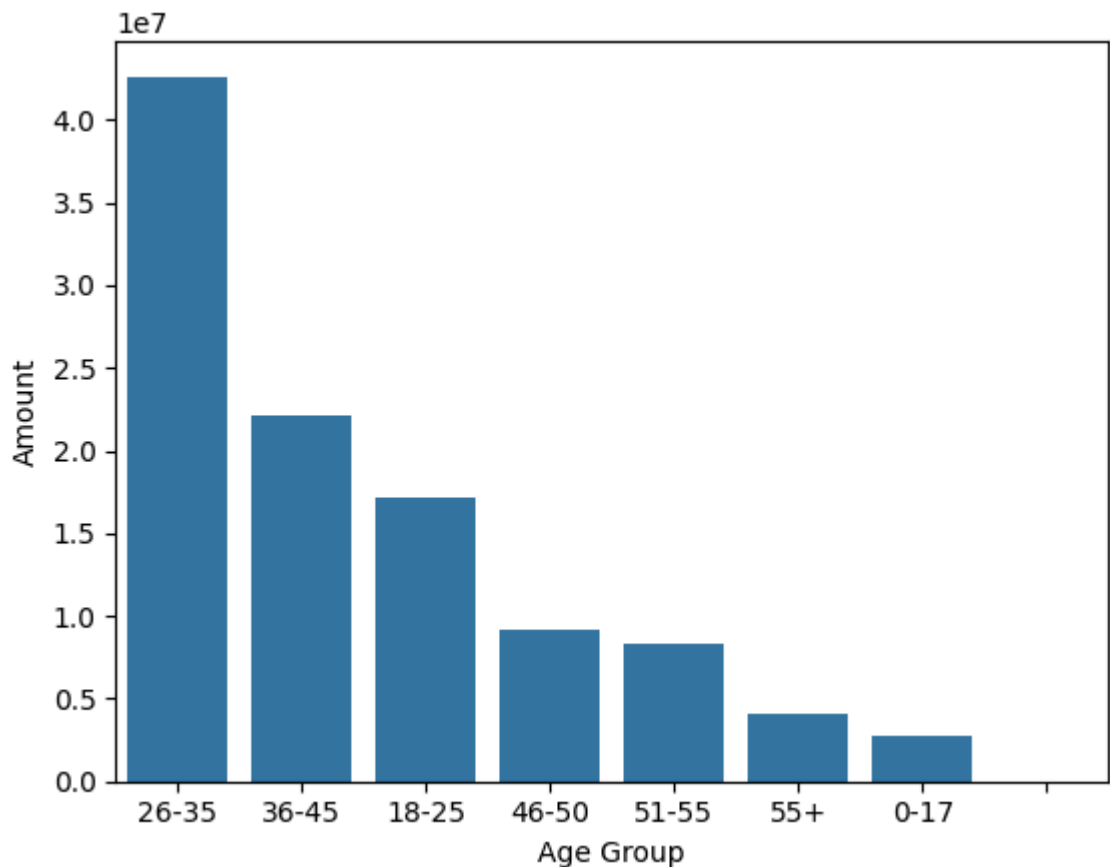
```
In [23]: ax = sns.countplot(data = df, x = 'Age Group', hue = 'Gender')  
  
for bars in ax.containers:  
    ax.bar_label(bars)
```



In [24]: *# Total Amount vs Age Group*

```
sales_age = df.groupby(['Age Group'], as_index=False)['Amount'].sum().sort_values('Amount', ascending=False)
sns.barplot(x = 'Age Group', y= 'Amount' ,data = sales_age)
```

Out[24]: <Axes: xlabel='Age Group', ylabel='Amount'>



Observation

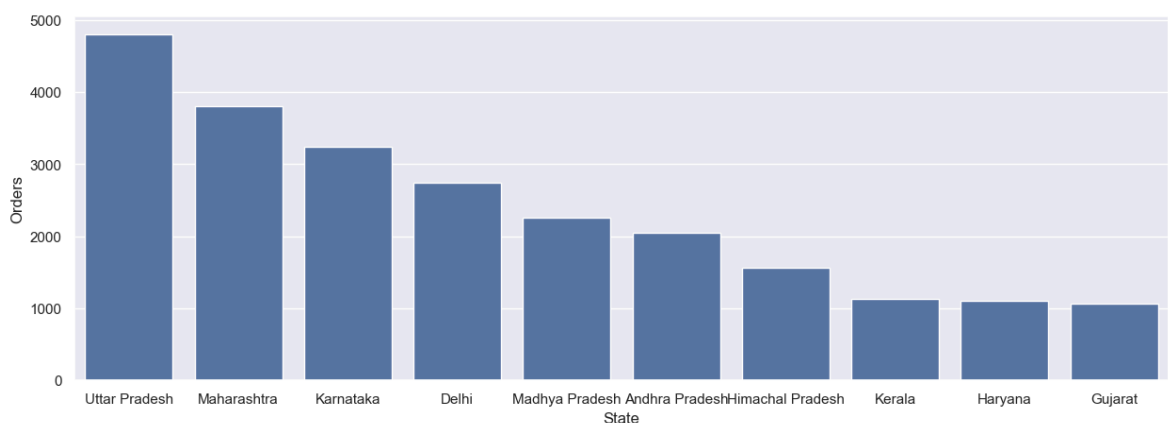
The majority of customers belong to the 26–35 age group, indicating that young adults contribute significantly to total sales.

State-wise Sales Analysis

```
In [25]: # Total number of orders from top 10 states
sales_state = df.groupby(['State'], as_index=False)['Orders'].sum().sort_

sns.set(rc={'figure.figsize':(15,5)})
sns.barplot(data = sales_state, x = 'State',y= 'Orders')
```

Out[25]: <Axes: xlabel='State', ylabel='Orders'>

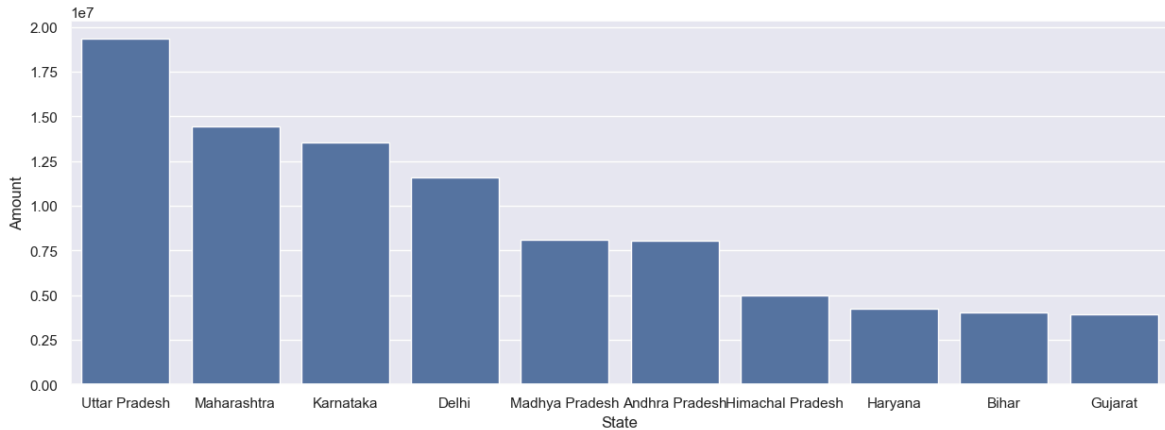



```
In [26]: # total amount/sales from top 10 states

sales_state = df.groupby(['State'], as_index=False)['Amount'].sum().sort_

sns.set(rc={'figure.figsize':(15,5)})
sns.barplot(data = sales_state, x = 'State',y= 'Amount')
```

Out[26]: <Axes: xlabel='State', ylabel='Amount'>



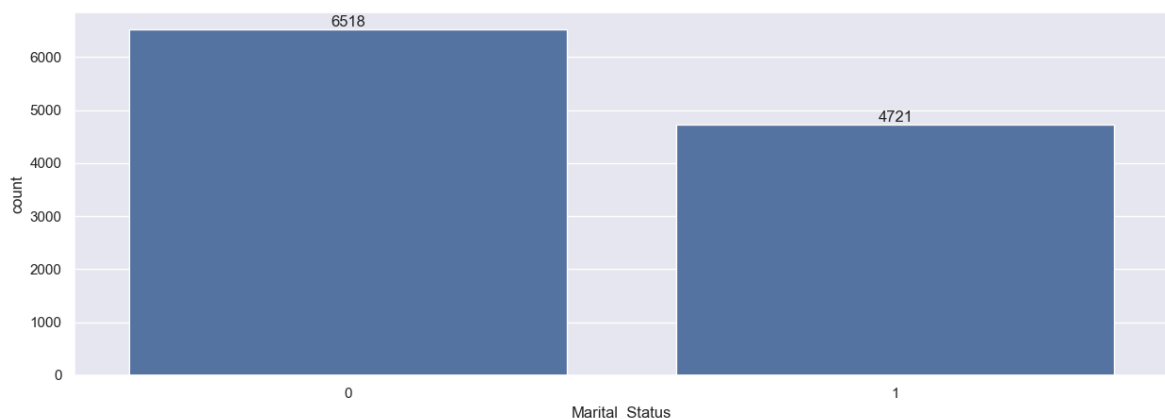
Observation

Uttar Pradesh, Maharashtra, and Karnataka generated the highest number of orders and total sales.

Marital Status Analysis

```
In [27]: ax = sns.countplot(data = df, x = 'Marital_Status')

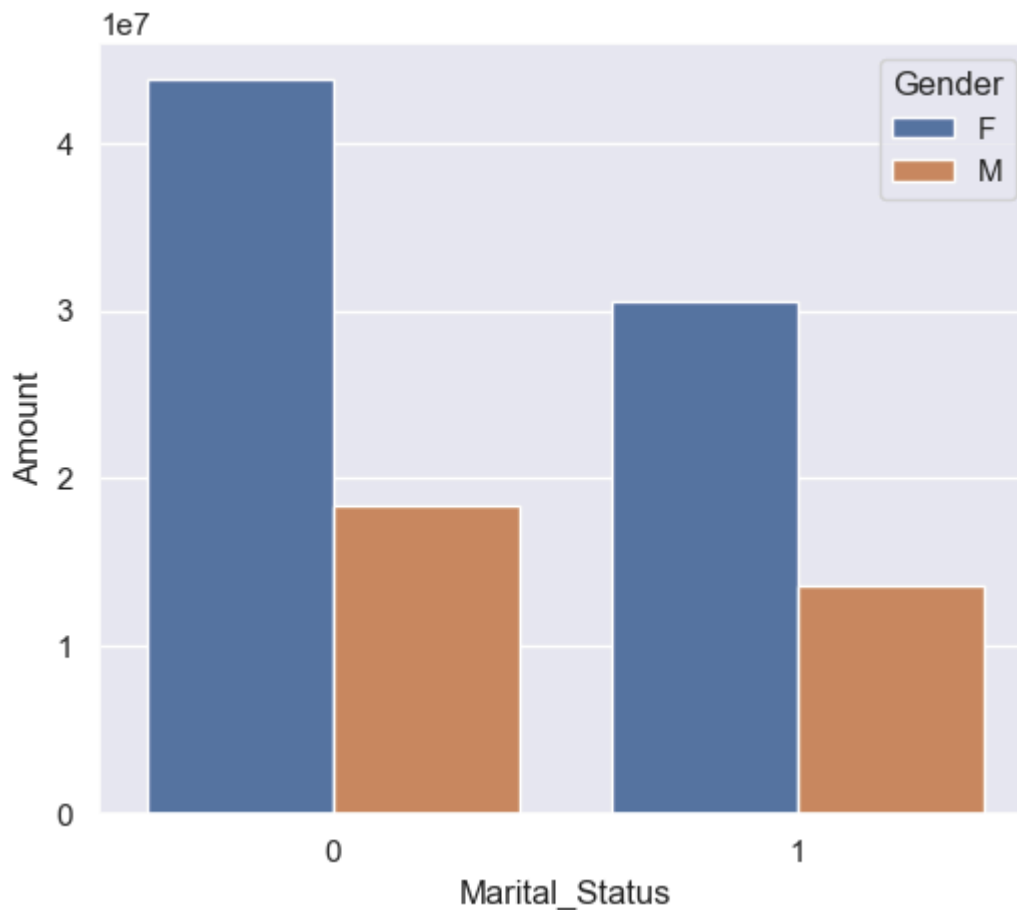
sns.set(rc={'figure.figsize':(7,5)})
for bars in ax.containers:
    ax.bar_label(bars)
```



```
In [28]: sales_state = df.groupby(['Marital_Status', 'Gender'], as_index=False)['A

sns.set(rc={'figure.figsize':(6,5)})
sns.barplot(data = sales_state, x = 'Marital_Status',y= 'Amount', hue='Ge
```

Out[28]: <Axes: xlabel='Marital_Status', ylabel='Amount'>



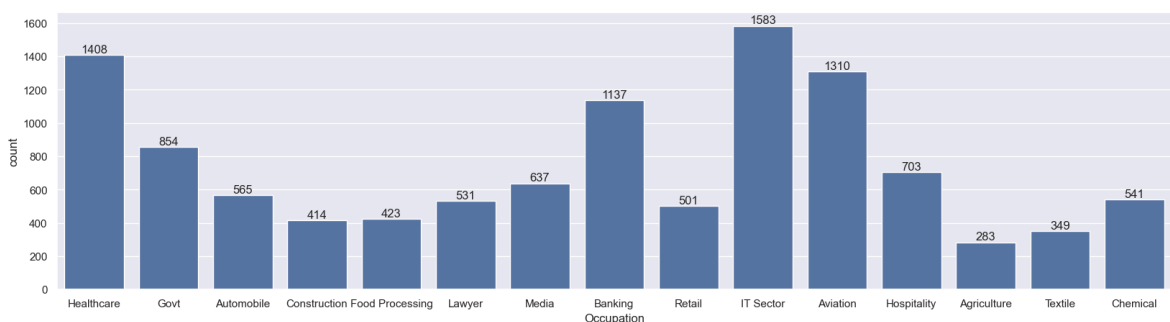
Observation

Married women contribute the highest purchasing power among all customer segments.

Occupation Analysis

```
In [29]: sns.set(rc={'figure.figsize':(20,5)})
ax = sns.countplot(data = df, x = 'Occupation')

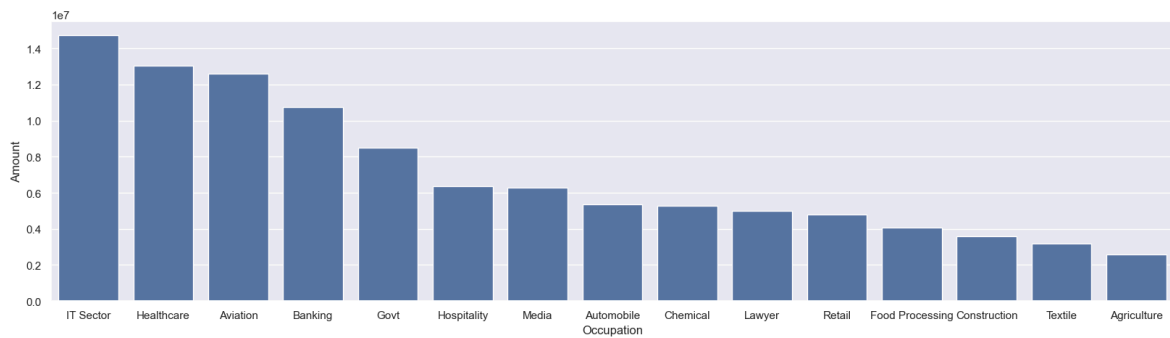
for bars in ax.containers:
    ax.bar_label(bars)
```



```
In [30]: sales_state = df.groupby(['Occupation'], as_index=False)['Amount'].sum()

sns.set(rc={'figure.figsize':(20,5)})
sns.barplot(data = sales_state, x = 'Occupation', y = 'Amount')
```

Out[30]: <Axes: xlabel='Occupation', ylabel='Amount'>



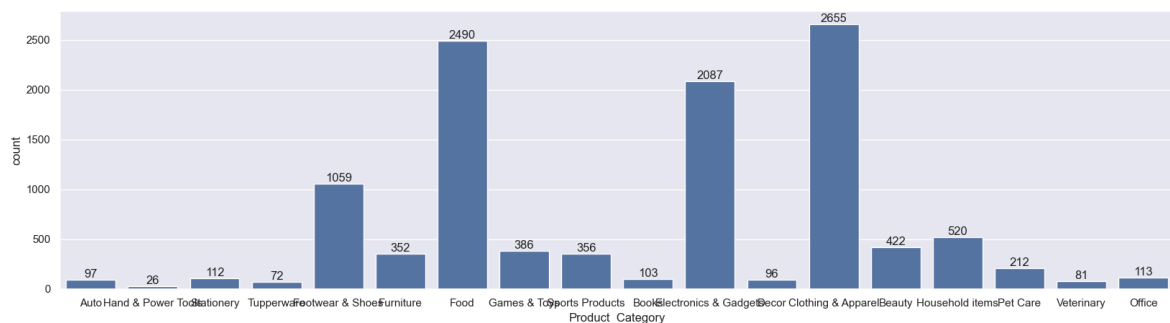
Observation

Customers working in IT, Healthcare, and Aviation recorded the highest purchase amounts.

Product Category Analysis

```
In [31]: sns.set(rc={'figure.figsize':(20,5)})
ax = sns.countplot(data = df, x = 'Product_Category')

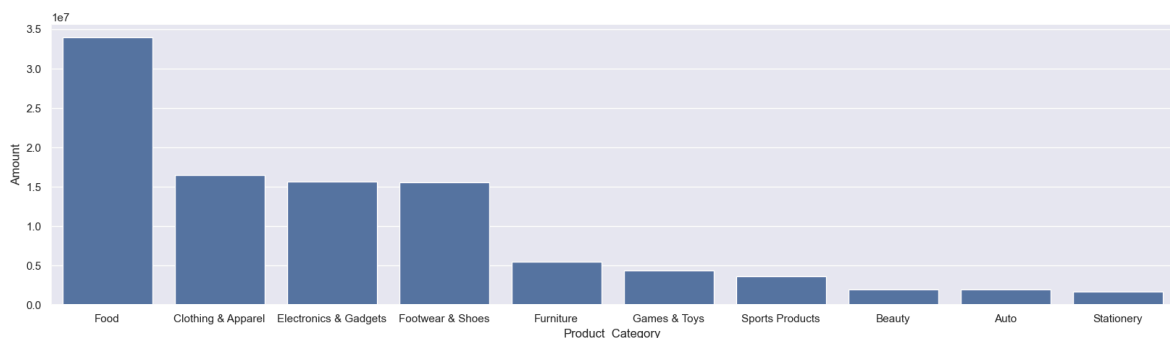
for bars in ax.containers:
    ax.bar_label(bars)
```



```
In [32]: sales_state = df.groupby(['Product_Category'], as_index=False)['Amount'].

sns.set(rc={'figure.figsize':(20,5)})
sns.barplot(data = sales_state, x = 'Product_Category', y= 'Amount')
```

Out[32]: <Axes: xlabel='Product_Category', ylabel='Amount'>



Observation

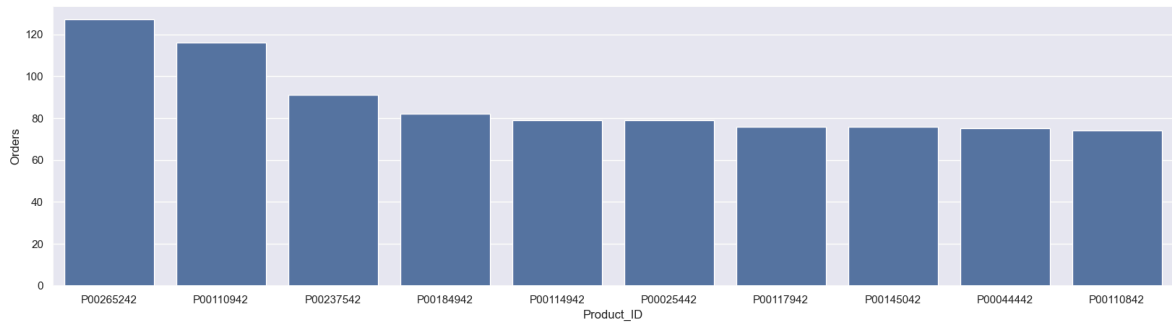
Food, Clothing, and Electronics are the most popular product categories based on sales and purchase volume.

Top Selling Products

```
In [33]: sales_state = df.groupby(['Product_ID'], as_index=False)['Orders'].sum().

sns.set(rc={'figure.figsize':(20,5)})
sns.barplot(data = sales_state, x = 'Product_ID',y= 'Orders')
```

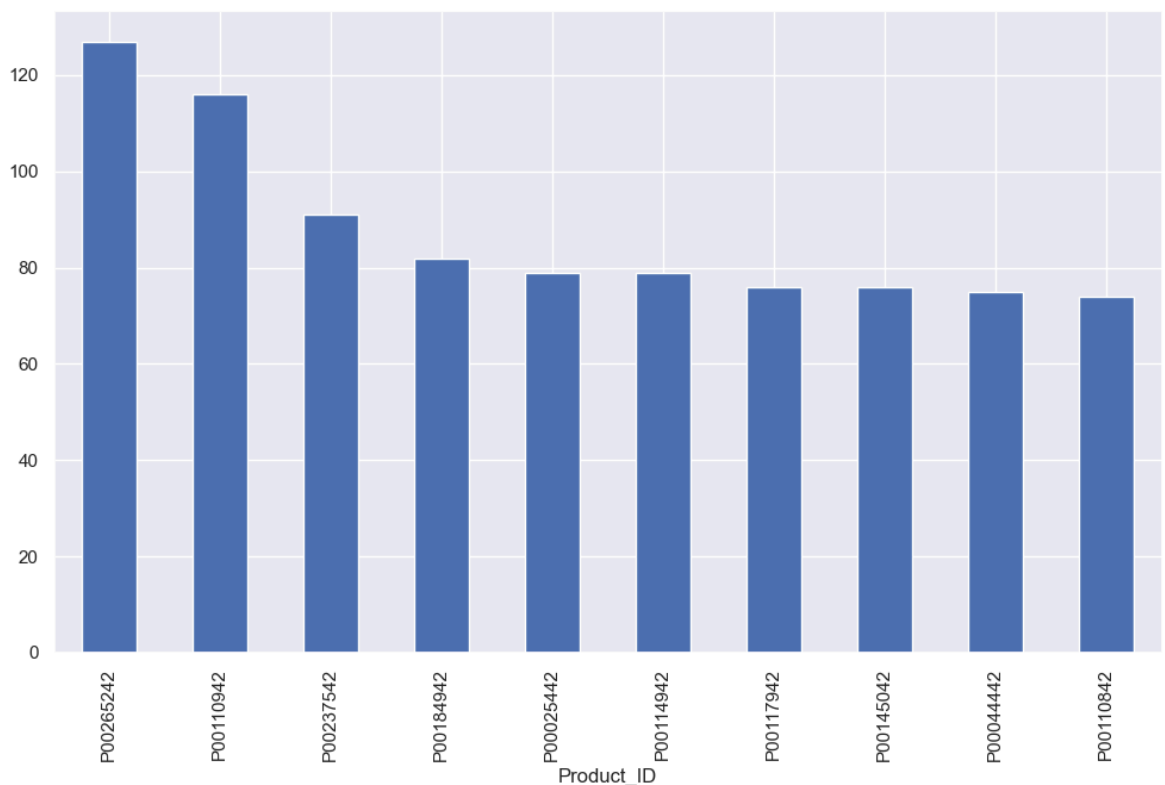
Out[33]: <Axes: xlabel='Product_ID', ylabel='Orders'>



```
In [34]: # top 10 most sold products

fig1, ax1 = plt.subplots(figsize=(12,7))
df.groupby('Product_ID')['Orders'].sum().nlargest(10).sort_values(ascending=False)
```

Out[34]: <Axes: xlabel='Product_ID'>



Observation

The analysis identifies the products with the highest order volume, helping businesses understand customer preferences.

Key Insights

- Female customers contributed the highest overall sales.
 - Customers aged 26–35 represented the largest purchasing segment.
 - Uttar Pradesh, Maharashtra, and Karnataka generated the highest sales.
 - IT, Healthcare, and Aviation professionals showed the highest spending.
 - Food, Clothing, and Electronics were the top-performing product categories.
-

Conclusion:

The analysis highlights key customer segments, regional sales trends, and product preferences during the Diwali season. These insights can support targeted marketing strategies, inventory planning, and data-driven business decisions.

End of Report

Prepared by: Shreya Saneja

Project: Diwali Sales Analysis using Python